Lecture 8

Terraform for Data Pipelines

Finalizing: Automating the provisioning of cloud infrastructure for data pipelines

Tutorial: Hands-on lab — Deploying baseline infrastructure (storage buckets, databases) with Terraform

Reference: Brikman, Terraform: Up and Running, 3rd Ed. — Chapters 4, 5, 6, 7

Recap: Terraform for Data Pipelines

- **Terraform** provisions and wires the **“where”**: servers, containers, databases, object storage, queues
- **Airflow** orchestrates the **“when”** and **“what”**: DAGs, tasks, scheduling
- From Lectures 6–7: Docker webservers, `depends_on`, variables, outputs, state

Today: Go deeper — modules, loops, conditionals, secrets, multiple providers, and a real baseline infra lab.

Today's Roadmap

1. **Chapter 4** — Reusable infrastructure with Terraform modules
2. **Chapter 5** — Loops, if-statements, deployment, gotchas
3. **Chapter 6** — Managing secrets with Terraform
4. **Chapter 7** — Working with multiple providers
5. **Hands-on lab** — Deploy baseline infra (buckets + database)

Chapter 4

Reusable Infrastructure with Terraform Modules

What Are Modules?

A **module** is a reusable, configurable unit of Terraform configuration.

- Encapsulates a set of resources (e.g. “S3 bucket + IAM policy”)
- Has **inputs** (variables) and **outputs**
- Can be called multiple times with different parameters
- Reduces duplication and standardizes patterns

```
module "data_bucket" {
  source = "./modules/s3-bucket"
  name   = "my-data-pipeline-raw"
  env    = "prod"
}
```

Creating a Basic Module

Structure:

```
modules/
  s3-bucket/
    main.tf      # resources
    variables.tf # inputs
    outputs.tf   # outputs
```

Calling the module:

```
module "raw_bucket" {
  source = "./modules/s3-bucket"
  bucket_name = "my-pipeline-raw"
}
```

Terraform treats `source` as the root of the module; resources inside are prefixed with `module.<name>.<resource>`.

Module Inputs and Outputs

variables.tf (inside module):

```
variable "bucket_name" {
  description = "S3 bucket name"
  type        = string
}
variable "env" {
  type    = string
  default = "dev"
}
```

outputs.tf (inside module):

```
output "bucket_id" {
  value       = aws_s3_bucket.this.id
  description = "S3 bucket ID"
}
```

Local Values

`locals` define computed values used across the module (or root config):

```
locals {
  common_tags = {
    Project = "data-pipeline"
    Env     = var.env
    Managed = "terraform"
  }
  bucket_name = "${var.project}-${var.env}-${var.suffix}"
}

resource "aws_s3_bucket" "this" {
  bucket = local.bucket_name
  tags   = local.common_tags
}
```

Cleaner than repeating expressions; evaluated once per run.

Versioned Modules

Use version constraints when sourcing from registries (Terraform Registry, Git, etc.):

```
# Terraform Registry
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 5.0"
  # ...
}

# Git
module "bucket" {
  source = "git::https://github.com/org/repo.git?ref=v1.2.0"
}
```

Semantic versioning: `~> 5.0` = `>= 5.0, < 6.0`

Module Gotchas

- **Count/for_each in modules** — Pass them from the caller; don't use inside child modules for "create/don't create" logic.
- **Provider inheritance** — Child modules use the provider of the caller; use `provider` meta-argument to override.
- **State** — Modules don't have separate state; all resources end up in the root state.
- **Refactoring** — Moving resources in/out of modules can cause destroy+recreate; use `moved` or `terraform state mv`.

Chapter 5

Loops, If-Statements, Deployment & Gotchas

Loops: count

Create multiple instances of a resource:

```
resource "aws_s3_bucket" "buckets" {
  count  = 3
  bucket = "my-bucket-${count.index}"
}
```

- `count.index` = 0, 1, 2
- **Limitation:** Adding/removing items in the middle changes indices → can cause unexpected destroys/recreates.

Loops: for_each

Better for dynamic sets/maps; uses keys for identity:

```
resource "aws_s3_bucket" "buckets" {
  for_each = toset(["raw", "staged", "curated"])
  bucket   = "my-pipeline-${each.key}"
}

# With map
resource "aws_s3_bucket" "by_env" {
  for_each = { dev = "dev-bucket", prod = "prod-bucket" }
  bucket   = each.value
}
```

- `each.key` and `each.value`
- Changing the set/map only affects added/removed items.

for and for String Directive

for expression (produces list/map):

```
locals {
  bucket_names = [for s in ["raw", "staged"] : "my-${s}-bucket"]
  # ["my-raw-bucket", "my-staged-bucket"]
}
```

for string directive (interpolation):

```
output "ips" {
  value = "%{for ip in aws_instance.web.*.public_ip}${ip}, %{endfor}"
}
```

Conditionals: count

“Create or don’t create”:

```
resource "aws_instance" "optional" {
  count  = var.create_bastion ? 1 : 0
  ami    = data.aws_ami.amazon_linux.id
  # ...
}
```

Access: `aws_instance.optional[0]` (only if count = 1). Be careful with `count` for conditional resources — refactoring can be tricky.

Conditionals: for_each and if

for_each with empty map:

```
resource "aws_instance" "bastion" {
  for_each = var.create_bastion ? { "0" = {} } : {}
  # ...
}
```

if in for expression:

```
locals {
  prod_only = [for k, v in var.config : v if v.env == "prod"]
}
```

if string directive:

```
output "msg" {
  value = "Prod: %{if var.env == "prod"}yes%{else}no%{endif}"
```

Zero-Downtime Deployment

- **create_before_destroy** — Create new resource before destroying old (e.g. launch config + ASG):

```
lifecycle {
  create_before_destroy = true
}
```

- **ignore_changes** — Don't update in-place when certain attributes change (e.g. AMI managed elsewhere):

```
lifecycle {
  ignore_changes = [ami]
}
```

Common Gotchas

- **Valid plan can fail** — API limits, quotas, permissions, transient errors. Use `-parallelism=1` or retry.
- **count/for_each limits** — Can't use both on same resource; switching between them usually requires `terraform state mv`.
- **Refactoring** — Use `moved` blocks (Terraform 1.1+) to avoid destroy/recreate:

```
moved {
  from = aws_s3_bucket.old
  to   = aws_s3_bucket.new
}
```

Chapter 6

Managing Secrets with Terraform

Types of Secrets

- **Provider auth** — AWS keys, GCP service account, Azure credentials
- **Resource attributes** — DB passwords, API keys, connection strings
- **State & plans** — `terraform.tfstate` and `plan` files can contain secrets in plain text

Storing Secrets: Options

Approach	Pros	Cons
Env vars	Simple, no files	Need to set before each run
Encrypted files	SoX, GPG	Key management
Vault / AWS SM / Azure KV	Centralized, audit	Extra infra, provider setup
IAM roles / OIDC	No long-lived keys	Cloud-specific

Provider Authentication

Environment variables (recommended for local/dev):

```
export AWS_ACCESS_KEY_ID=...
export AWS_SECRET_ACCESS_KEY=...
export AWS_SESSION_TOKEN=...  # if using SSO
```

IAM roles (EC2, ECS, Lambda): No credentials in code; instance/execution role is used automatically.

OIDC (CI/CD): GitHub Actions, GitLab CI can assume AWS roles via OIDC — no static keys.

Secrets in Resources

Avoid:

```
resource "aws_db_instance" "db" {
  password = "hardcoded-bad"  # Never do this
}
```

Prefer:

```
variable "db_password" {
  type      = string
  sensitive = true
}
# Set via: TF_VAR_db_password=... or -var="db_password=..."
```

Or use **external secret stores** (Vault, AWS Secrets Manager) with data sources and avoid passing secrets through Terraform at all when possible.

State and Plan Security

- **State** contains resource attributes, including secrets. Use:
  - Remote backend (S3) with encryption
  - Restrict access (IAM, bucket policy)
- **Plan files** (`-out=tfplan`) can include secrets. Don't commit them; treat as sensitive.
- **Sensitive outputs:** Use `sensitive = true` so they're not shown in logs:

```
output "db_endpoint" {
  value     = aws_db_instance.db.endpoint
  sensitive = true
}
```

Chapter 7

Working with Multiple Providers

How Providers Work

- **Install:** `terraform init` downloads providers from the registry.
- **Version:** Pin in `required_providers`:

```
required_providers {
  aws = { source = "hashicorp/aws", version = "~> 5.0" }
}
```

- **Configure:** `provider "aws" { ... }` with region, profile, etc.

Multiple Copies of the Same Provider

Multiple regions:

```
provider "aws" {
  alias  = "us-east"
  region = "us-east-1"
}
provider "aws" {
  alias  = "eu-west"
  region = "eu-west-1"
}

resource "aws_s3_bucket" "replica" {
  provider = aws.eu-west
  bucket   = "my-bucket-eu"
}
```

Multiple accounts: Use different `profile` or `assume_role` per provider alias.

Multiple Different Providers

Use several providers in one config:

```
provider "aws" { region = "us-east-1" }
provider "google" { project = "my-gcp-project" }
provider "datadog" { api_key = var.datadog_api_key }

resource "aws_s3_bucket" "data" { ... }
resource "google_storage_bucket" "backup" { ... }
```

Common for cross-cloud or hybrid setups (e.g. AWS + GCP, cloud + Datadog).

Reusable Modules with Multiple Providers

Pass provider into the module:

```
module "bucket_us" {
  source   = "./modules/s3-bucket"
  providers = {
    aws = aws.us-east}
  bucket_name = "my-bucket-us"
}
module "bucket_eu" {
  source   = "./modules/s3-bucket"
  providers = {
    aws = aws.eu-west}
  bucket_name = "my-bucket-eu"
}
```

The module declares `required_providers` and optionally uses `provider` meta-argument in resources.

Lab Objectives

Deploy a **baseline infrastructure** for a data pipeline using Terraform:

1. **S3 buckets** — e.g. `raw`, `staged`, `curated` (or equivalents)
2. **Database** — e.g. RDS MySQL/PostgreSQL (or DynamoDB for simpler lab)
3. **Outputs** — Bucket names, DB endpoint for use by Airflow/pipelines

Location: `lecture8/hands-on-lab/`

Lab Structure

```
lecture8/hands-on-lab/
├── main.tf          # Providers, modules/resources
├── variables.tf     # region, project, env
├── outputs.tf       # bucket IDs, DB endpoint
├── backend.tf       # (optional) S3 remote state
└── README.md        # How to run
```

Use **variables** for `project`, `env`; use **for_each** or **count** to create multiple buckets if you like.

Lab: Quick Start

```
cd lecture8/hands-on-lab
terraform init
terraform plan
terraform apply
```

Outputs to verify:

- S3 bucket names (raw, staged, curated)
- RDS endpoint (if using RDS)
- Security group IDs

Cleanup:

```
terraform destroy
```

Summary

Summary

- **Modules** — Reusable, configurable units with inputs/outputs; version from registry or Git.
- **Loops & conditionals** — `count`, `for_each`, `for`, `if`; prefer `for_each` for dynamic collections.
- **Secrets** — Env vars, IAM roles, OIDC; avoid hardcoding; use `sensitive = true`; protect state.
- **Multiple providers** — Aliases for regions/accounts; pass providers into modules.
- **Hands-on** — Baseline infra (buckets + DB) as foundation for data pipelines.

Next Steps

- Complete the hands-on lab in `lecture8/hands-on-lab/`
- Try wrapping the lab in a **module** for reuse across dev/stage/prod
- Integrate Terraform outputs with Airflow Variables/Connections
- Reference: *Terraform: Up and Running*, 3rd Ed. — Chapters 4–7

Questions?